

1. System Overview

This document defines the functional and timing requirements for a digital logic block that receives a continuous 8 Mbit/s serial input stream, identifies the bit position of a single '1' bit within each frame, and outputs an extended frame containing the original 128-bit data followed by an 8-bit binary representation of the detected bit position.

The logic can make use of internal 64MHz design clock.

2. Input Interface

2.1 Input Clock

- Port name of input clock in testbench DUT: in_clk
- Frequency: **8 MHz** (8000 kHz)
- Source: external serial interface
- Clock is continuous with no missing pulses.
- Duty-cycle specification: **50% ±10%**
- Refer further to chapter about clock synchronization

2.2 Input Data

- Port name of serial input data in testbench DUT: in_data
- Data rate: **8 Mbit/s**, one data bit per rising clock edge
- Serial data is aligned to the input clock.
- Bit sampling shall occur on the rising edge with defined setup and hold times.

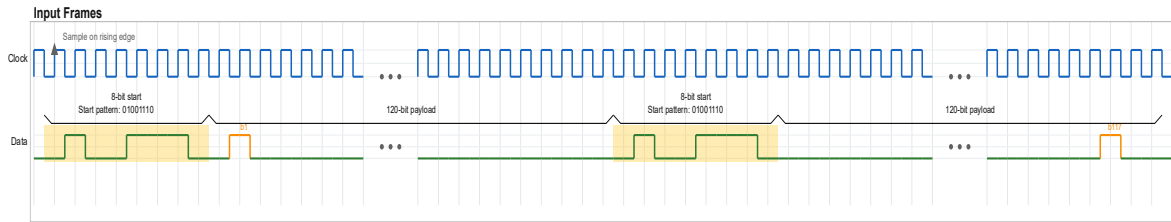
2.3 Input Frame Structure

Each frame consists of:

- **Frame Start Pattern (8 bits):** fixed pattern 01001110
 - The time order of frame bits in serial line is left-most bit first, right most last
- **Payload (120 bits):**
 - Contains **exactly one bit equal to '1'** and 119 bits equal to '0'
 - The index of first bit after frame start pattern is zero. The index of the last bit is 119.

Frames repeat in input data continuously with **no gaps** between them.

You can assume the input data structure is always valid and conforms to this specification.



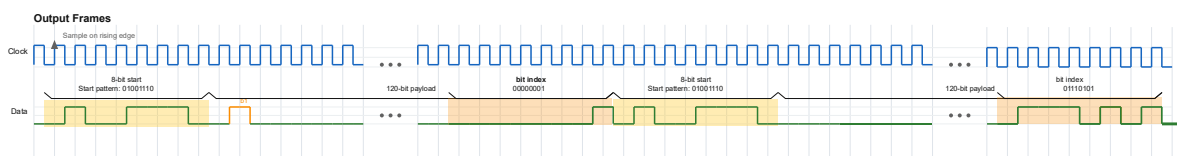
3. Output Interface

3.1 Output Clock

- Port name of output clock in testbench DUT: out_clk
- Must be synchronous with the input clock. Refer to chapter about clock synchronization.
- Frequency must be in proper relation to the input clock.
- Duty-cycle requirement: **50% ±20%**.

3.2 Output Data

- Port name out serial output data in testbench DUT: out_data
- Bit stream is continuous with **no gaps** between frames
- Output frame structure:
 - **Original 128-bit input frame**, unchanged
 - **Extended Field (8 bits)**: binary representation of the '1' bit position (0–119). Binary representation is sent MSB first.
- Total output frame length: **136 bits** (128 + 8)
 - All input frames which come as continuous bitstream without gaps must be sent to continuous (extended) output stream without gaps
- The maximum latency from serial data input to serial data output is two frames, 256 input clock cycles.



4. System clock Interface

- 64MHz system clock, rising edge to be used for clocking.
 - Port name of system clock in testbench DUT: clk64
 - Asynchronous reset, active low.
 - Port name of system clock in testbench DUT: rst_n
-

5. Internal Processing Requirements

5.1 Bit Position Detection

The logic shall:

- Identify the index of the single '1' bit within the 120-bit payload.
- Generate an 8-bit unsigned binary number representing that index (range 0–119).

5.2 Clocking Environment

- The design can use a **64 MHz internal system clock**.
 - Port name of system clock in testbench DUT: clk64
 - There is active low asynchronous reset signal for DUT
 - Port name of asynchronous reset in testbench DUT: rst_n
 - All internal operations, including frame parsing, buffering, and output serialization, can operate relative to this clock. Input data is relative to input clock. Output data is relative to output clock, each data bit should change on falling edge of output clock and setup and hold times of output data are relative to rising edge of output clock.
-

6. Timing Requirements

6.1 Input Timing

Input timing requirements shall be defined as:

- setup and hold requirements relative to the input clock
 - **Setup time** (data stable before clock edge), 5ns
 - **Hold time** (data stable after clock edge), 5ns

6.2 Output Timing

Output data must meet:

- setup and hold requirements relative to the output clock
 - **Setup time** (data stable before clock edge), 5ns
 - **Hold time** (data stable after clock edge), 5ns
 -
 - Output clock duty-cycle must be within **50% ±20%**.
 - continuous streaming, no gaps on clock or data
-

7. Clock Synchronization Requirements

The output clock must be **synchronous** to the input clock:

- Input 8 MHz clock is synchronous to system 64 MHz clock.
 - CDC protection is not required for the purposes of this exercise
 - There must be no long-term frequency drift between input and output data stream.
 - Phase alignment must remain constant within jitter tolerance.
 - Output clock must be free running with continuous pulse train.
 - Output frame is continuous frame stream, no gaps between frames.
 - There is fixed latency between input frame and output frame, less than 2 frames
 - Output clock should meet specified duty cycle and jitter requirements
-

10. Test environment overview

Here are instructions for using simple environment with testbench which tests the functional operation and timing of DUT.

Testbench provides 64MHz design internal clock, input data clock and serial input data. It checks output data clock and output data from DUT.

11. Instructions to run

There are scripts to compile and run the design with the provided testbench. They first remove temporary files left from previous runs, then compile the DUT (via a participant-provided file) and the provided testbench Verilog files. After compilation, either a batch simulation or an interactive Verdi session is started.

Before running script, VCS and Verdi must be ready to use in shell where script is run.

When the simulation runs, the testbench drives clocks and input data to the DUT, checks the DUT outputs, and prints either a *[PASS]* or *[ERROR]* result. Script output first shows compilation messages; near the end of the simulation you should see messages such as:

- [PASS] Checked XXX frames successfully.
- \$finish called from file "tbdir/tb_serial_frame_extender_sub.sv.e", line 1.
- \$finish at simulation time nnnnnnnnnnnnnnn
-

or

- [ERROR] Got output frame with no reference.
- \$finish called from file "tbdir/tb_serial_frame_extender_sub.sv.e", line 1.
- \$finish at simulation time nnnnnnnnnnnnnnn
-

The provided testbench top file *tb/tb_serial_frame_extender_top.sv* instantiates the DUT top module *serial_frame_extender* and the encrypted testbench module *tb_serial_frame_extender_sub*, and connects signals between them. The DUT top must keep the same module/entity name and the same interface as in the template files. You may modify a template or build a new file hierarchy, including mixed VHDL and Verilog, as long as the top name and interface stay the same.

What you must provide/edit:

- Your DUT RTL files (you may start from the templates in the provided *vhdl/* and *verilog/* folders)
- the existing file *compiledut* in the same directory as *runbatch* / *rungui* - you do not need to create this file; it is already provided with comments and commented examples. Adapt it for your design (uncomment and edit the relevant lines when you add, rename, or move sources).

compiledut contains only the Synopsys analyze commands for all DUT sources. Paths are relative to the directory from which *runbatch* (or a GUI script) is run. Subdirectories (e.g. *vhdl/*, *verilog/*) are allowed. If you add, rename, or move files, update *compiledut* and list files in a valid compile order.

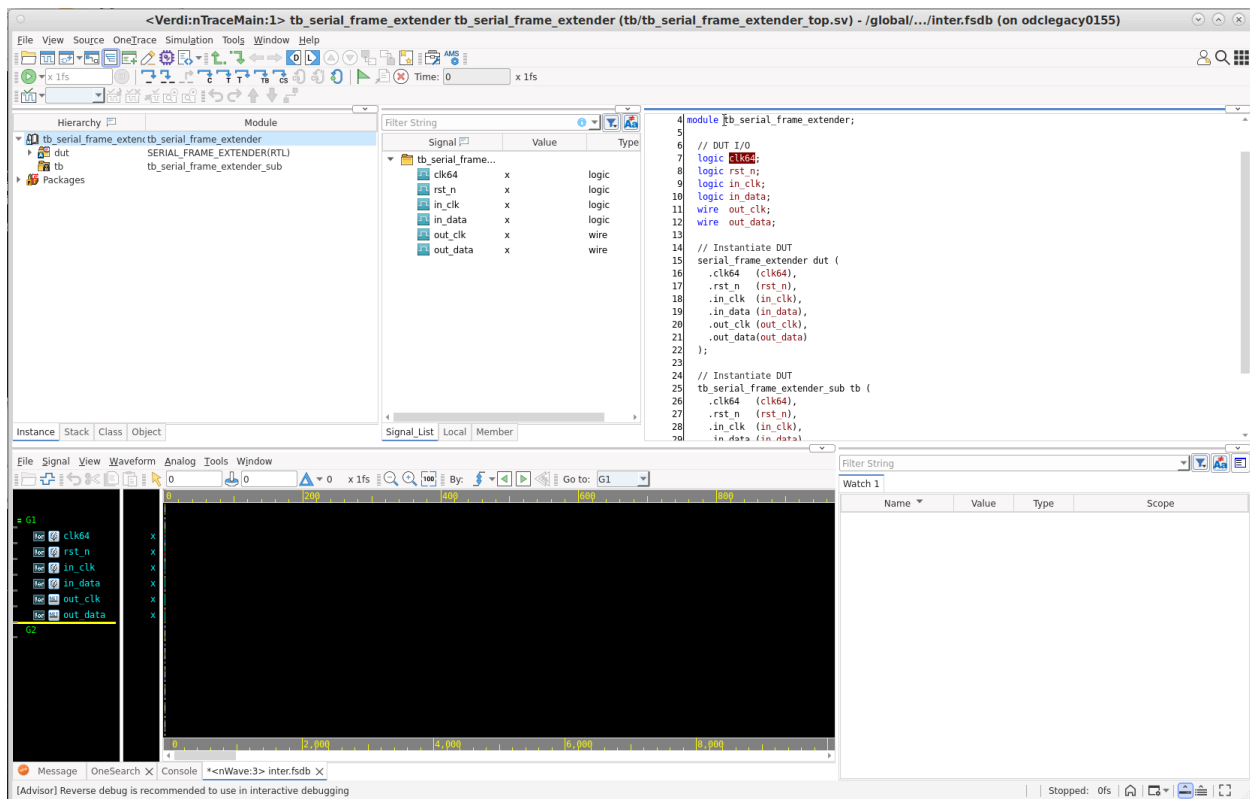
Do not modify *runbatch*, *rungui*, or the testbench files.

11.1 batch run with *runbatch*

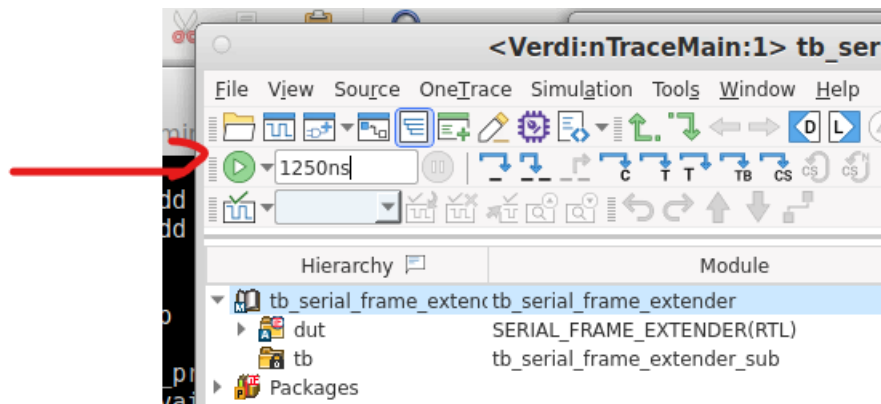
- *runbatch* is an executable script. Run it from the shell:
 - `./runbatch`
- This script cleans temporary files, sources *compiledut* to analyze the DUT, then compiles the provided testbench and runs the simulation in batch mode
- Results appear in the shell output ([PASS] / [ERROR] as above).

11.2 GUI run with *rungui*

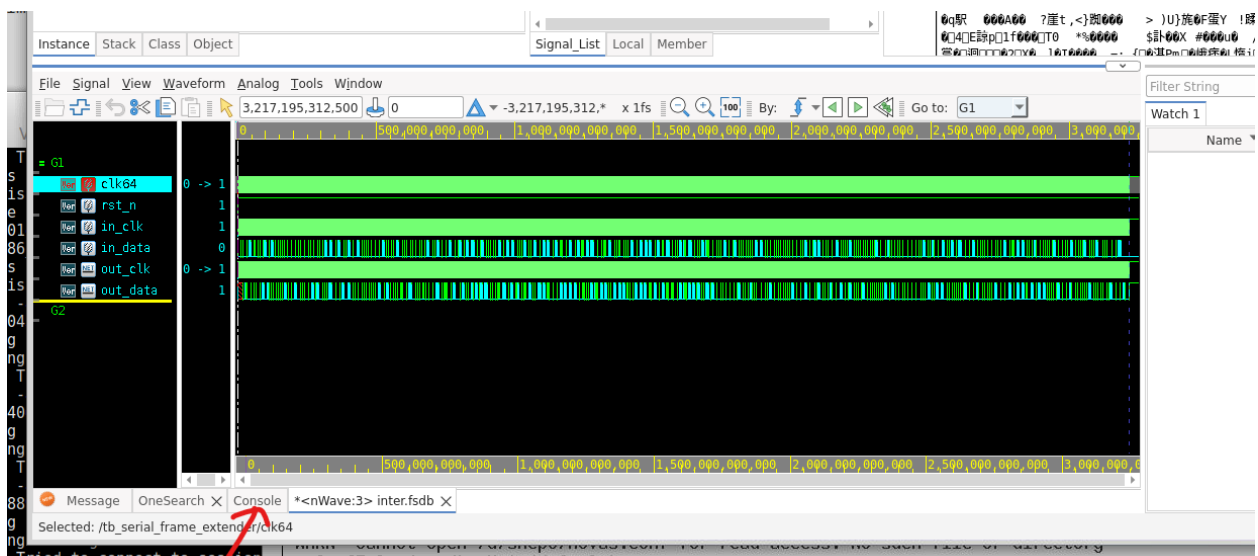
- Same flow as 2.1 for compilation (also sources *compiledut*), but opens Verdi for debug.
- Verdi starts with the provided *dustrun.ses* session file.
- *dustrun.ses* sets up the waveform window with DUT input and output signals. Below is snapshot of window that is expected to be seen after running script from shell command line.



- You can browse the instance hierarchy in *Instance* window pane, which is above black *Waveform* pane. **Double clicking** name in *Instance* pane will change content of signal list in *Signal_List* pane and source code in source code pane. These window panes are located right of the *Instance* pane.
- Most of the items in UI have context sensitive menu by clicking right mouse button over the item (instance name, signal name in *Signal_List* or *Waveform* or source code...)
- Copy and paste work between window panes, you can also experiment Drag-and-Drop between window panes.
- You can add signals you want to debug into *Waveform* pane either by Drag-and-Drop or context sensitive right-click menu (or keyboard shortcut) from *Signal_List* and source code panes.
- Running simulation can be started with green icon in upper left corner. You can set the length of time to run in the box located on the left of icon, or leave box empty to run into end of simulation



- File drop-down menu, top left in picture above, contains *Restore Session* and *Save Session* commands, which can be used to save and restore modified window layouts and signals added to waveform between sessions.
- Simulation can be re-run without recompile from *Simulation* drop-down menu.
- Window panes have several Tabs. One important Tab is *Console* under Waveform



- Clicking Console opens tab where you can give simulator commands, but most importantly you should click this after running the simulation (in this example to the end of simulation) to see the simulation messages:

Instance Stack Class Object Signal_List Local Member

Rule Time View Tools Window

3,217,195,312,500 x1fs Line: 13 Rule: /UVM_OVM_i_rule.rc

10 500,000,000,000 1,000,000,000,000 1,500,000,000,000 2,000,000,000,000 2,500,000,000,000 3,000,000,000,000

```
13
14 *Verdi* : Create FSDB file '/global/gtseu06/user/shepo/frame_extender/verilog/build_hack_v4enc/inter.fsdb'
15
16 *Verdi* : Flush all FSDB Files at 0 fs.
17
18 *Verdi* : Enable RPC Server(3563822)
19
20 srcTBInvokeSim
21 Verdi>fsdbDumpvars 0 "tb_serial_frame_extender.clk64" +all +trace_process;fsdbDumpvars 0 "tb_serial_frame_extender.rst_n" +all +trace_process;f
22 *Verdi* : Dumping the signal (tb_serial_frame_extender.clk64).
23 *Verdi* : Dumping the signal (tb_serial_frame_extender.rst_n).
24 *Verdi* : Dumping the signal (tb_serial_frame_extender.in_clk).
25 *Verdi* : Dumping the signal (tb_serial_frame_extender.in_data).
26 *Verdi* : Dumping the signal (tb_serial_frame_extender.out_clk).
27 *Verdi* : Dumping the signal (tb_serial_frame_extender.out_data).
28 Verdi>run
29 [PASS] Checked 200 frames successfully.
30 $finish called from file "tbdir/tb_serial_frame_extender_sub.sv.e", line 1.
31 $finish at simulation time 3217195312500
```

Verdi>

Message OneSearch Console *<nWave:3> inter.fsdb X

Selected: /tb_serial_frame_extender/clk64

tried to connect to session guiConfFile (read)= N/A (default)